

46-926, Fall 2020: Homework #5

Due **3:20pm**, December 9, 2020.

Homework should be submitted electronically on canvas before the deadline. Please submit your homework as a single file (pdf preferred, canvas struggles with html). We recommend using jupyter notebooks to prepare your homework.

Do all computational problems in python unless otherwise noted. Please post any questions on the Piazza discussion board.

Remarks: The majority of this homework focuses on understanding classification metrics, losses, and ROC curves. We'll use the marketing data set and model introduced in problem 1 for the rest of the problems as an example for exploring these ideas.

1. *Prediction with marketing data* For this problem, we will be working with marketing data from a bank, found in `marketing.csv` on Canvas. We observe the results from a calling campaign that attempted to sell a new product to clients. Based on the covariates of the client, we want to build a classifier to predict whether the client will invest in the product.

We observe the following variables about the client:

- `age`
- `job`: Job classification
- `marital`: Marital status
- `education`: Level of education
- `default`: Currently in default?
- `balance`: Average yearly account balance
- `housing`: Currently has a housing loan?
- `loan`: Currently has a personal loan?

You also observe a variable `y` which is either `no` or `yes`, indicating whether the sale was successful. This is your outcome variable.

You can load the data into `python` with the `read_csv` command. Familiarize yourself with the variables and data types in this data set.

Hint: You will notice that we have several categorical variables. In sklearn's logistic regression, these are not handled automatically like they are in R. Instead, you need to convert them to dummy variables (also called one-hot encoding). You can do this by calling the `pd.get_dummies` function on your data frame, which returns a dummy-encoded version of the data frame.

Similarly, you can recode your `y` variable as $\{0, 1\}$ with

```
y = np.where(dat['y'] == 'yes', 1, 0)
```

or just use a column returned from the dummy call.

Make sure to do all of your recoding of variables before you split the samples into train and test.

We will also split the data into a training set and a test set. For uniformity and ease of grading, use the following code to generate your training and test data:

```
train_test_split(X, y, test_size=0.33, random_state=1)
```

This splits off 33% of the data to be used as a validation set.

- (a) Using the training data, estimate the fraction of successes (your *base rate*). This will be useful to keep in mind.
- (b) Using your training data, fit a logistic regression using all of the variables to predict the outcome. Use `LogisticRegression` from `sklearn`, with `penalty='none'` to down-weight the ridge penalty. What is your misclassification rate (fraction of wrong guesses in any direction) on the test set, using the usual logistic regression classification rule?

Hint: You can get three different kinds of “predictions” from logistic regression. Suppose that `fit` is the object returned by a call to `LogisticRegression().fit()`. Then:

- `fit.predict` gives class labels in $\{0, 1\}$
- `fit.predict_proba` gives probabilities on the $p(x)$ scale. Note that this returns probabilities of both class 0 and class 1, so
`fit.predict_proba(X_test)[: , 1]`
corresponds to the vector of $P(Y = 1|X = x_i)$.
- `fit.predict_log_proba` similarly gives the log probabilities. This is sometimes more convenient.

- (c) If you used the silly classifier that guesses ‘no’ for all observations, what would your misclassification rate be on the test data? How does this compare to the misclassification rate of your logistic classifier?

This might be rather discouraging for our classifier, if our true goal is minimizing 0-1 loss.

- (d) Suppose that, despite this hopelessness, you still feel the need to call some clients. Using your logistic model, find the 1000 clients in the test set that are most likely to score ‘yes’. (That is, the 1000 observations from the test set with the highest predicted probability of $Y = 1$.) For this set of 1000 clients, what fraction of them actually say yes? What fraction would you expect if you picked a random set of 1000 clients?

Suddenly our model feels a little more useful!

Hint: You may find the `np.argsort` and/or `np.flip` functions useful. You’re welcome to do this in other ways, though.

- (e) Even though our simple classifier doesn’t win in terms of 0-1 loss, we see that it helps to sort our points according to their likelihood of being ‘yes’. This can be a much more reasonable goal than misclassification error in heavily imbalanced samples. You can think of this as an approach to selecting an “enriched” set. As you change the size of your selected set (500, 1000, 2000, etc), you change both:

- The fraction of true positives in the data that you manage to capture (called *sensitivity*)
- The fraction of true negatives you manage to avoid (called *specificity*)

When the set gets larger, sensitivity will necessarily increase, but specificity will decrease. The better your classifier, the more efficiently it makes this trade off.

An easy way to assess the quality of your classifier in this sense is to plot sensitivity versus specificity. A better classifier will tend to have a higher curve (better sensitivity for a given specificity). These curves (called ROC curves) have other nice properties that we will discuss in class; This problem is just intended as an introduction.

The `roc_curve` function from `sklearn.metrics` will compute TPR (the same as sensitivity) and FPR (the same as 1-specificity). It returns a tuple, of which the first two elements are FPR and TPR. Example usage:

```
fpr, tpr, _ = roc_curve(true_y_values, predicted_probabilities)
```

(The `_` discards the third returned value, which we don't care about yet)

Using the `roc_curve` function, plot the TPR (y axis) against the FPR (x axis). This is your ROC curve.

If you classified by guessing randomly, on average you would just get the diagonal $y = x$ line; if your classifier is better than random guessing it should give a curve above this line. Include a diagonal $y = x$ line in your plot for comparison.

2. Fun with ROC curves and AUC!

Begin with the marketing data set and the logistic model that you fit earlier.

For each point in the test data, compute the predicted probabilities. From these probabilities, compute the logit transformation:

$$\log\left(\frac{p(x_i)}{1 - p(x_i)}\right)$$

These are equivalent to your $x_i^T \hat{\beta}$ scores from the logistic regression. Plotting scores on this scale will make for nicer pictures. It won't change the ROC curve or the classifications from thresholding, since the logit is just a monotonic function. These transformed logit scores will be what I mean by "scores" below.

- (a) First, just make a histogram of your logit scores.

Hint: The plot and axis objects have a `hist` method. You can set the number of bins with the `bins` argument.

- (b) Now make separate histograms of the score for observations from each class. Stack these two histograms vertically in the same figure. Make sure to title each with the correct class, and to force the x -axis limits to be the same on both plots.

You should see that the score has a different distribution for each class of observation. We are able to use our score to classify because the distribution of the score for class 'yes' tends to be larger than the distribution of the score for class 'no'.

Hint: You can set the x limits with the `range` argument of `hist`, which takes a two-element list or tuple.

You can set titles with the `set_title` method of the axis object.

- (c) Suppose that we let $f_0(x)$ denote the distribution of the score for class ‘no’, and $f_1(x)$ denote the distribution of the score for class ‘yes’. Furthermore, suppose we decide to threshold our scores at cutoff T , and classify all points with score $> T$ into class ‘yes’.

Write out the True Positive Rate (TPR) and False Positive Rate ($FPR = 1 - \text{specificity}$) in terms of integrals of these two distributions, f_0, f_1 . Redraw your two histograms, this time adding in a vertical line at $T = -1.5$. What part of the new plots correspond to the True Positive Rate and False Positive Rate? (You can describe this in words since labeling it in your pdf might be hard.)

Hint: You can add vertical lines with the `axvline` method of the axis object.

- (d) It is convenient to think about your score in terms of these class distributions. It will also let us prove a nice fact about ROC curves!

Write out the Area Under the Curve (AUC) in terms of an integral involving the $TPR(T)$ and $FPR(T)$.

Hints:

Realize that you are looking at a curve parametrized by $(FPR(T), TPR(T))$.

Remember from calculus: For a parametric curve $x=f(t)$, $y=g(t)$, the area under the curve is $\int_{t_0}^{t_f} g(t)f'(t)dt$. In your case, $t_0 = \infty$ and $t_f = -\infty$ (flipped because big T corresponds to small x -axis values).

- (e) Simplify this expression to get it in terms of $\mathbb{P}(S_1 > S_0)$, if S_0 were the score of a random point from class ‘no’ and S_1 were the score of a random point drawn from class ‘yes’.

When you succeed, you will have proven that the AUC is the probability that your score successfully ranks a random point from class 1 higher than a random point from class 0!

Hints:

As you simplify, try to introduce an indicator of the event $\{S_1 > S_0\}$. Remember that S_1 is drawn from f_1 and S_0 is drawn from f_0 .

3. *Do-it-yourself sensitivity-specificity.* We will do a little more exploration of ROC curves and loss in this problem. As an example, we again consider the scores from logistic regression on the marketing data set.

Note: All ROC curves in this problem should be produced by hand, rather than with a built-in python ROC function.

- (a) Write a function that takes in a boolean vector of guessed categories, a boolean vector of true categories, a loss for false positives and a loss for false negatives, and produces sensitivity, specificity, and total loss. You may find it useful to use boolean expressions like `(truth)&(~estimate)` in computing the desired quantities (where `~` is the elementwise “not” operator). The function should be laid out like this:

```
#This function should take in numpy logical arrays for estimate
#and truth and single values for the losses
def eval_estimate(estimate, truth, loss_FP, loss_FN):
    #Things that you write go here
    return(sens, spec, loss)
```

To help us verify your function, run it on the scores from logistic regression thresholded at -1. Set $L_{FP} = 5$ (loss for a false positive) and $L_{FN} = 100$ (loss for a false negative) as in class, here and for the remainder of this problem.

- (b) Apply your function to all the possible thresholds in your problem. These are just the midpoints between each pair of logit values. This gives all possible classifications that can be obtained by your scores. Make a plot showing all of the sensitivity-specificity pairs that you obtain. To get a curve that looks like a typical ROC curve, you’ll actually want to plot $1 - \text{specificity}$ on the x -axis. Add a diagonal line to compare to.

You may use the following code to produce breakpoints and apply your function if you like:

```
unique_values = np.sort(np.unique(logits))
midpoints = (unique_values[0:(len(unique_values)-1)]+
             unique_values[1:len(unique_values)])/2.0
sens, spec, loss = np.vectorize( lambda x: eval_estimate(logits>x,
                np.array(y_test, dtype=bool), 5, 100))(midpoints)
```

- (c) Color the points on your previous plot by the total loss that you computed for each point. Given a vector of `values`, you can color your points by those values in python with code like

```
from matplotlib import cm #For nice colors
plt.scatter(xvalues, yvalues, c=values, cmap=cm.inferno)
```

This will show you how your loss varies over your curve (with red values being better losses). I recommend using the negative of your loss values as the color values, so that bright colors indicate good losses.

- (d) Find the minimum loss point on your curve (see `np.argmin`). Figure out its sensitivity and specificity. Plot your roc curve again without colors, and plot this minimizing point on top in a different color.

Draw a line on top of this curve which passes through that point and has slope given by $\frac{N}{P} \cdot \frac{L_{FP}}{L_{FN}}$, where N, P are the total number of negative and positive observations in the test set, and L_{FP} and L_{FN} are the False Positive and False Negative losses.

What do you notice?

Hint: To add a point to a plot, just call `scatter` with a single coordinate. You can set the color with `color` and scale the point size with `s`. You may have to make `s` quite large to be visible.

```
plt.scatter(x_coord, y_coord, color='green', s=200)
```

Hint: Plotting lines by slope and intercept is weirdly annoying in python. I stole this code from the internet to make it easier:

```
def abline(slope, intercept):
    """Plot a line from slope and intercept"""
    axes = plt.gca()
    x_vals = np.array(axes.get_xlim())
    y_vals = intercept + slope * x_vals
    plt.plot(x_vals, y_vals, '--')
```

(source <https://stackoverflow.com/a/43811762>)

- (e) Prove this formula!
- i. Write out the total loss as a function of N, P , the False Positive Rate (FPR), the False Negative Rate (FNR), and L_{FP}, L_{FN} .
 - ii. Rewrite FPR and FNR as integrals of f_0 and f_1 , as in Problem 2c, with some set threshold T .
 - iii. Take a derivative of this expression with respect to T and set it equal to 0. Solve for $f_1(T)/f_0(T)$, the slope of the tangent to the parametric curve at the minimizing point.
- (f) In your last homework, you derived the optimal threshold on the true probabilities for a given set of losses. Continuing with your logistic regression and from the last problem and the same losses (5 and 100), compute the optimal probability threshold for this problem.
- What is your loss at this optimal threshold? How does it compare to the minimum loss that you found in part d? (**Hint:** You can reuse your `eval_estimate` function.)
- (g) Starting with your ROC curve from earlier in this problem, add one more point for the sensitivity/specificity of your ideal point. How does it compare to the point you already plotted for your true minimum? Give a reason why these might not agree completely, and why the loss for your ideal classifier threshold is not actually the best.
- (h) **REMOVED.** There was a part h on early parts of this problem. It should have been removed, since it was dependent on the ungraded problem at the end of this assignment. You do not need to do it.

UNGRADED BELOW THIS

Here is one more problem that I've decided not to assign, out of concern for assignment length. However, you might enjoy(?) doing it when you have a break.

1. *Marketing data, using additive models.* Now we will try to improve your marketing classifier. Start where you left off in problem 1, using the same data set.

The point of this problem is to show you that the same additive model approach that we used in regression can also be used in logistic regression for classification. This gives us a way to incorporate variables non-linearly.

- (a) Two of our variables, balance and age, are continuous. Furthermore, we might doubt that the log odds of successfully advertising grows linearly in either of these variables. Fit a logistic GAM to the data, using smoothing splines to model the effect of balance and age. Plot the partial dependence plots for these two variables. (Everything is essentially the same using `pygam`, except now you use `LogisticGAM`.)
- (b) Evaluate the predictions from your logistic GAM in terms of misclassification. Also calculate the fraction of successes in the top ranked 1000 test observations. How does your performance on each of these metrics compare to logistic regression?
- (c) Make a new ROC curve plot using the built in function for ROC curves. This plot should have three curves: (i) the straight $y = x$ line; (ii) The ROC curve for logistic regression from the previous problem; (iii) The ROC curve for your new logistic GAM.